Instituto Tecnológico de Software
CCT 31PSU0097H

Tecnológico de Software



Tecnológico de Software

FUNDAMENTOS DE ARQUITECTURA DE SOFTWARE

TSU: **INNOVACIÓN Y DESARROLLO DE SOFTWARE**

ALUMNOS: **JESÚS UC – ÁNGELA ROJAS – ÁNGEL LUGO**

DOCENTE: **GIMER AMILCAR CERVERA EVIA**

GRUPO: **4-B (Dual)**

FECHA DE ENTREGA: **26/10/26**

EXPOSICIÓN POR EQUIPOS

REDIS: CACHÉ EN MEMORIA PARA UNA API

Instituto Tecnológico de Software
CCT 31PSU0097H

REDIS: CACHÉ EN MEMORIA PARA UNA API

INTRODUCCIÓN

En una aplicación backend, no todas las solicitudes necesitan volver a consultar la base de datos desde cero. Cuando varios usuarios piden la misma información, repetir las consultas puede aumentar el trabajo del servidor y el tiempo de respuesta. Redis es una herramienta que puede ayudar en ese escenario, ya que permite guardar datos en memoria y recuperarlos rápidamente. En este reporte se explica qué problema resuelve, cómo se integra con una API, cómo se relaciona con MySQL y Docker, y qué ventajas y límites conviene considerar.

¿QUÉ ES REDIS Y QUÉ PROBLEMA RESUELVE?

Redis es un almacén de datos en memoria que se utiliza, entre otras cosas, como caché. Una caché conserva temporalmente información que puede volver a necesitarse, de modo que la aplicación no tenga que calcularla o consultarla de nuevo cada vez. Por ejemplo, si una API muestra una lista de productos populares y muchas personas la solicitan, la aplicación puede guardar el resultado en Redis y reutilizarlo durante un periodo.

El objetivo es reducir consultas repetidas a la base de datos y mejorar el tiempo de respuesta cuando el caso de uso se beneficia de ello. Redis no garantiza que cualquier aplicación vaya a ser más rápida: el resultado depende de qué datos se almacenen, de la frecuencia de las solicitudes y del costo de consultar la fuente original.

COMPONENTES Y FUNCIONAMIENTO GENERAL

En un ejemplo básico intervienen el cliente, la API, Redis y MySQL. El cliente solicita información a la API; la API revisa primero si el resultado está disponible en Redis. Si lo encuentra y sigue siendo válido, lo devuelve. Si no está, consulta MySQL, obtiene los datos, guarda temporalmente una copia en Redis y responde al cliente.

1. El cliente envía una petición HTTP a la API.
2. La API busca en Redis una clave asociada a la información solicitada.
3. Si hay un resultado vigente, la API lo devuelve sin repetir la consulta a MySQL.
4. Si no hay resultado, la API consulta MySQL y obtiene los datos originales.
5. La API guarda el resultado en Redis con un tiempo de expiración definido y responde al cliente.
6. Cuando los datos cambian, la aplicación debe actualizar o invalidar la caché para evitar que se use información antigua.

Este patrón se conoce como caché de lectura. El tiempo de expiración depende del tipo de información: los datos que cambian con frecuencia suelen necesitar periodos más cortos o una estrategia de invalidación más cuidadosa.

EJEMPLO APLICADO A UNA API DE PRODUCTOS

Supongamos que el catálogo de una tienda tiene el endpoint GET /productos/populares. Cada visita requiere mostrar una lista de productos consultados con frecuencia. Sin caché, la API podría consultar MySQL en cada petición. Con Redis, se puede usar la clave productos:populares para almacenar temporalmente la lista.

El flujo sería el siguiente: la API busca productos:populares en Redis; si encuentra la lista y no ha expirado, la devuelve. Si no la encuentra, consulta MySQL, obtiene los productos, guarda la lista en Redis —por ejemplo, durante 60 segundos— y devuelve la respuesta. Las siguientes solicitudes dentro de ese tiempo pueden reutilizar el resultado. Los 60 segundos son solo un ejemplo; el equipo debe elegir un plazo adecuado para los datos reales.

Si un producto cambia de precio o deja de estar disponible, el sistema debe definir cómo actualizar o borrar la entrada de caché. De lo contrario, la API podría mostrar información desactualizada hasta que expire. Este detalle es importante porque una caché mejora el acceso, pero también requiere una estrategia de actualización.

RELACIÓN CON MYSQL Y DOCKER

Redis frente a MySQL

MySQL se utiliza habitualmente como base de datos relacional principal, donde se conservan los datos organizados en tablas y se realizan consultas y operaciones persistentes. Redis trabaja con estructuras de datos accesibles en memoria y suele utilizarse para caché, contadores, sesiones y otros datos de acceso rápido. En el ejemplo, MySQL mantiene los datos del catálogo y Redis guarda una copia temporal de una consulta frecuente. Por eso pueden complementarse en lugar de reemplazarse.

Redis y Docker

Redis es el servicio que almacena y entrega los datos; Docker es una herramienta para empaquetar y ejecutar aplicaciones y servicios en contenedores. Un equipo puede ejecutar Redis en un contenedor durante el desarrollo para facilitar su instalación y configuración. Docker no realiza la función de caché por sí mismo y Redis no sustituye la función de Docker.

VENTAJAS Y LIMITACIONES

Ventajas

- Puede reducir el tiempo de respuesta de consultas repetidas.
- Puede disminuir la carga sobre la base de datos principal.
- Permite reutilizar datos temporales, como resultados, sesiones o contadores.
- Puede incorporarse a una API existente sin reemplazar necesariamente la base de datos.

Limitaciones

- Los datos en caché ocupan memoria, que es un recurso limitado.
- Una caché mal actualizada puede devolver información antigua.
- Añade configuración, monitoreo y mantenimiento al sistema.
- No todas las consultas se benefician: si los datos cambian constantemente o casi no se repiten las solicitudes, la mejora puede ser pequeña.

¿CUÁNDO CONVIENE UTILIZARLO Y CUÁNDO NO?

Redis puede ser una buena opción cuando hay información que se consulta muchas veces, el resultado puede reutilizarse durante un periodo y reducir la latencia o la carga de la base de datos aporta un beneficio claro. También puede ser útil cuando se necesitan contadores o sesiones de acceso rápido.

No siempre es necesario añadir Redis a una aplicación pequeña o con pocas consultas repetidas, especialmente si la complejidad adicional no produce una mejora apreciable. Tampoco conviene tratar una caché como única fuente de información importante sin analizar las necesidades de persistencia, recuperación y consistencia de los datos.

CONCLUSIÓN

Redis es una herramienta útil para mejorar el acceso a información que se consulta repetidamente, siempre que se elijan bien los datos que se guardan y se controle cuánto tiempo permanecen en caché. En una API de catálogo, puede trabajar junto a MySQL: la base de datos conserva la información principal y Redis ayuda a responder más rápido algunas consultas frecuentes. La decisión de incorporarlo debe basarse en una necesidad concreta, porque también implica administrar memoria, expiración e invalidación de datos.

FUENTES CONSULTADAS

- Redis. (s. f.). Redis Documentation. https://redis.io/docs/latest/
- Redis. (s. f.). Redis. https://redis.io/
- Docker. (s. f.). Docker Docs. https://docs.docker.com/
- ¿Qué es Redis? (2026, 15 junio). IBM. https://www.ibm.com/mx-es/think/topics/redis
- ¿Qué es Redis? – (AWS). (s. f.). Amazon Web Services, Inc. https://aws.amazon.com/es/elasticache/what-is-redis/
- Base de datos de Redis | Google Cloud: https://cloud.google.com/discover/what-is-redis-database?hl=es-419